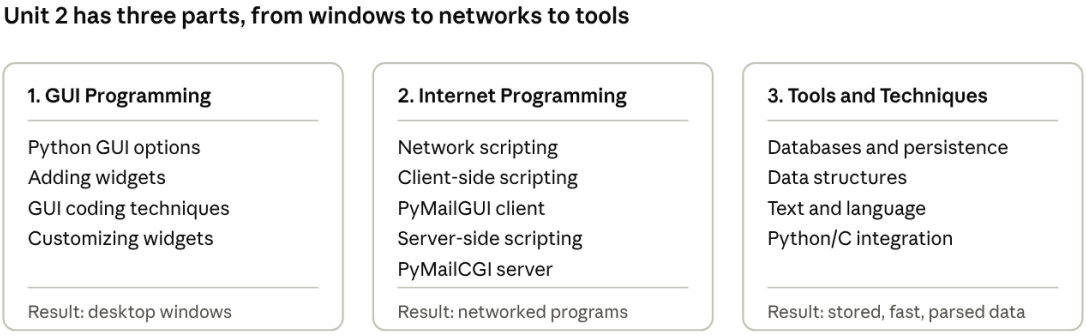
Data Science Unit 2: GUI, Internet Programming and Python Tools

Unit overview

Part	Syllabus topics	What you can build with it
1. GUI Programming	Python GUI options, adding widgets, GUI coding techniques, customizing widgets	A desktop window with buttons, text boxes and menus
2. Internet Programming	Network scripting, client-side scripting, PyMailGUI client, server-side scripting, PyMailCGI server	Programs that fetch pages, send mail or serve web pages
3. Tools and Techniques	Databases and persistence, data structures, text and language, Python/C integration	Saved data, efficient structures, text parsers and fast C-backed code



unit roadmap · 3 parts

PyMailGUI and PyMailCGI are case-study programs from Mark Lutz's *Programming Python*, a prescribed text book. This guide explains the ideas and Python tools those programs demonstrate; read the book's matching chapters for their full source.

1. GUI Programming

A graphical user interface (GUI) lets people use a program through windows, buttons and text boxes instead of typing commands; in Python the standard toolkit for this is Tkinter, and every GUI program works as an event loop that reacts to user actions.

1.1 Python GUI development options

Python has several GUI toolkits. Choose by licence, platform, look and how much you want to install.

Toolkit	Comes with Python?	Notes
Tkinter (with ttk)	Yes, in the standard library	Simple, portable and ideal for learning and small tools
PyQt / PySide	No (`pip install`)	Powerful Qt toolkit for large, polished desktop applications
wxPython	No	Uses the native widgets of each operating system
Kivy	No	Touch-friendly, runs on desktop and mobile
Web-based GUI (Flask, Streamlit, Dash)	No	The user interface runs in a browser instead of a desktop window

This unit uses **Tkinter** because it needs no installation, and its concepts (widgets, layout, events, callbacks) carry over to every other toolkit.

1.2 Adding widgets

A Tkinter program has three steps: create a **root window**, create **widgets** inside it and place them with a **geometry manager**, then start the **event loop** with `mainloop()`.

```
import tkinter as tk

root = tk.Tk()
root.title("Greeter")

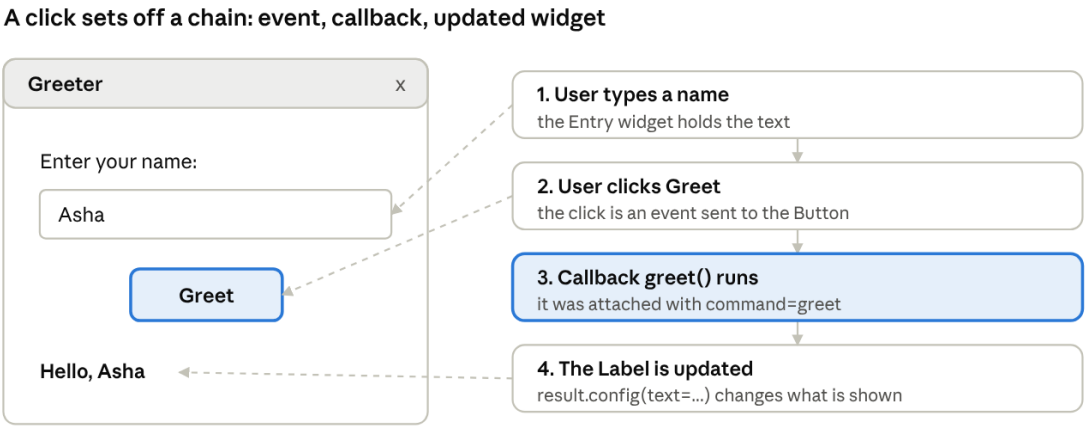
label = tk.Label(root, text="Enter your name:")
entry = tk.Entry(root, width=20)
result = tk.Label(root, text="")

def greet():                                    # callback function
    result.config(text="Hello, " + entry.get())

button = tk.Button(root, text="Greet", command=greet)

label.pack()
entry.pack()
button.pack()
result.pack()

root.mainloop()                                 # wait for events until the
window closes
```



Tkinter window and its event chain · 4 steps

The window on the left is what the program above draws; the cards on the right show what happens when the button is clicked.

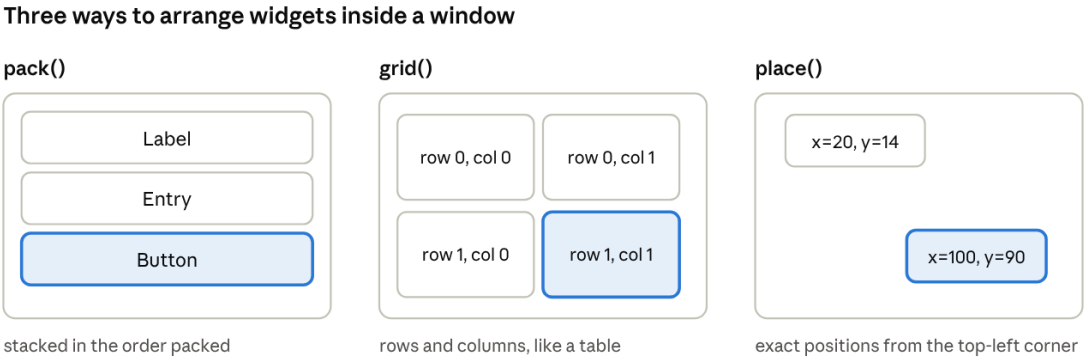
Widget	Purpose
`Label`	Shows text or an image
`Button`	Runs a callback when clicked
`Entry`	One-line text input
`Text`	Multi-line text area
`Frame`	Container used to group widgets
`Checkbutton` / `Radiobutton`	Tick box / one-of-many choice
`Listbox`	List of selectable items
`Scale` / `Spinbox`	Slider / number selector
`Canvas`	Area for drawing shapes, lines and images
`Menu`	Menu bar and pop-up menus
`Toplevel`	An extra window

Standard dialogs are ready-made: `tkinter.messagebox` (alerts and confirmations), `tkinter.filedialog` (open and save pickers) and `tkinter.colorchooser`.

Geometry managers decide where each widget sits. Use only one manager inside any one container.

Manager	Idea	Example
`pack`	Stack widgets against a side of the container	`w.pack(side="left", fill="x", padx=5)`

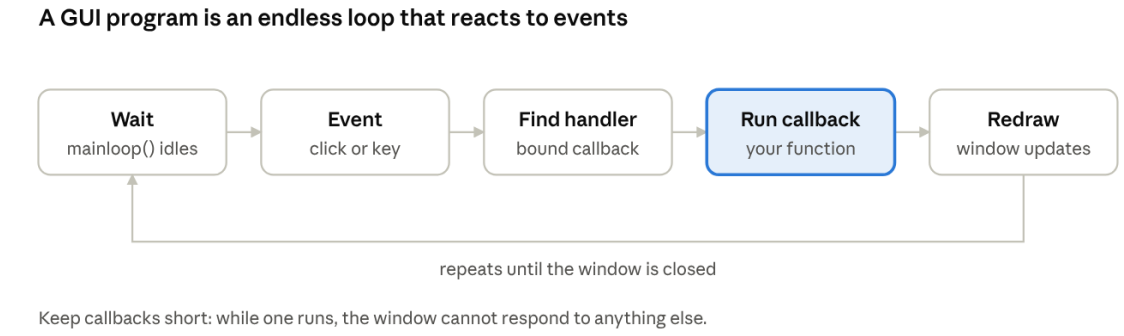
Manager	Idea	Example
`grid`	Place in rows and columns of a table	`w.grid(row=0, column=1, sticky="w")`
`place`	Exact x and y position	`w.place(x=20, y=40)`



pack, grid and place layouts

1.3 GUI coding techniques

- **Event loop and callbacks.** `mainloop()` waits for events such as clicks and key presses. Your function (the callback) runs when its event happens, and must return quickly or the window freezes.
- **Event binding.** `widget.bind("<Return>", handler)` attaches a function to any event; the handler receives an `event` object with details such as the key or mouse position.
- **Control variables.** `StringVar`, `IntVar` and `BooleanVar` keep a widget and your program in sync: `var = tk.StringVar(); tk.Entry(root, textvariable=var)`, then `var.get()` anywhere.
- **Class-based design.** Wrap a window in a class so state and callbacks live together.
- **Timers.** `root.after(1000, func)` schedules a call after 1000 ms without blocking.
- **Long tasks.** Never run a slow job (download, heavy calculation) inside a callback. Run it in a worker thread and pass results back through a `queue.Queue`, polling it with `after()`, because widgets must only be changed from the main thread.
- **Callbacks with arguments.** Use `lambda`: `Button(root, text="Add", command=lambda: add(5))`.



GUI event loop · 5 stages

```
import tkinter as tk
from tkinter import messagebox

class CounterApp(tk.Frame):
    def __init__(self, master):
        super().__init__(master)
        self.count = tk.IntVar(value=0)             # shared state
        tk.Label(self, textvariable=self.count, font=("Arial",
24)).grid(row=0, column=0, columnspan=2)
        tk.Button(self, text="+1", command=self.add).grid(row=1, column=0)
        tk.Button(self, text="Reset", command=self.reset).grid(row=1,
column=1)
        master.bind("<Escape>", lambda e: master.destroy())   # key binding
        self.pack(padx=20, pady=20)

    def add(self):
        self.count.set(self.count.get() + 1)
        if self.count.get() == 10:
            messagebox.showinfo("Counter", "You reached 10!")

    def reset(self):
        self.count.set(0)

root = tk.Tk()
CounterApp(root)
root.mainloop()
```

1.4 Customizing widgets

- **Options.** Most widgets accept `font`, `fg` (text colour), `bg` (background), `width`, `height`, `relief` (`flat`, `raised`, `sunken`, `groove`), `cursor`, `state` (`normal` or `disabled`) and `padx` / `pady`. Change them later with `widget.config(option=value)`.
- **Fonts and colours.** `font=("Helvetica", 14, "bold")`; colours are names (`"navy"`) or hex codes (`"#2a7ae2"`).
- **Images.** `img = tk.PhotoImage(file="logo.png")` then `Label(root, image=img)`; keep a reference to `img` or it disappears.

- **Themed widgets (ttk).** `from tkinter import ttk` gives modern-looking `ttk.Button`, `ttk.Entry`, `ttk.Combobox`, `ttk.Treeview` and `ttk.Notebook` (tabs); style them with `ttk.Style()`.
- **Custom widgets.** Subclass an existing widget to create a reusable component with its own defaults and behaviour.
- **Menus.** Build a menu bar with `tk.Menu` and `add_command(label=..., command=...)`.
- **Drawing.** `Canvas` methods such as `create_line`, `create_rectangle`, `create_oval` and `create_text` let you draw charts and games.

```
import tkinter as tk

class HoverButton(tk.Button):                       # a customized widget
    def __init__(self, master, **kw):
        super().__init__(master, bg="#e8e8e8", relief="groove",
font=("Arial", 12), **kw)
        self.bind("<Enter>", lambda e: self.config(bg="#cfe3ff"))   # mouse
over
        self.bind("<Leave>", lambda e: self.config(bg="#e8e8e8"))   # mouse
away

root = tk.Tk()
HoverButton(root, text="Hover over me", command=root.destroy).pack(padx=30,
pady=30)
root.mainloop()
```

2. Internet Programming

Internet programs follow the client-server model: a server waits for requests on a network address and port, and a client connects, sends a request and reads the reply; Python's standard library covers every layer from raw sockets to email and web pages.

2.1 Network scripting

A **socket** is one end of a two-way network connection, identified by an IP address and a port number. Higher-level protocols such as HTTP (web), SMTP (send mail), POP3 and IMAP (read mail) and FTP (file transfer) are built on top of sockets.

Module	What it does
`socket`	Low-level TCP and UDP connections
`socketserver`	Framework for writing servers that handle many clients
`urllib.request` / `http.client`	Fetch web pages and call web services
`smtplib`	Send email through an SMTP server
`poplib` / `imaplib`	Read email from a mailbox
`ftplib`	Upload and download files by FTP

Module	What it does
`email`	Build and parse email messages and attachments
`json`	Convert data to and from the text format used by web APIs

```
# server.py : a minimal TCP echo server
import socket

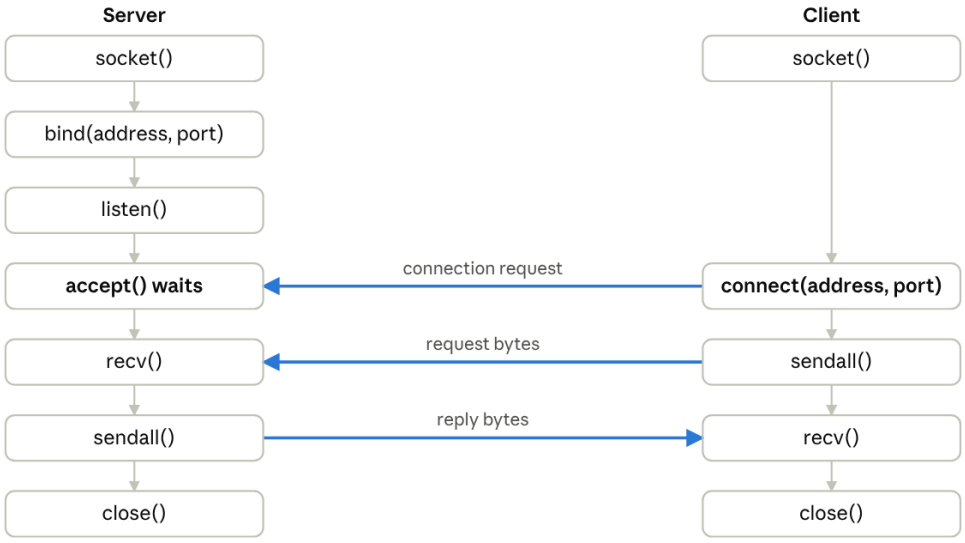
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)   # TCP socket
server.bind(("127.0.0.1", 5000))        # claim address and port
server.listen()                         # start waiting for clients
print("Waiting for a client...")
conn, addr = server.accept()            # blocks until a client connects
with conn:
    data = conn.recv(1024)              # read up to 1024 bytes
    conn.sendall(b"Echo: " + data)      # reply
server.close()
```

```
# client.py
import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect(("127.0.0.1", 5000))      # connect to the server
    s.sendall(b"Hello server")
    print(s.recv(1024).decode())        # Echo: Hello server
```

Sockets carry **bytes**, so text must be encoded (`"text".encode()`) before sending and decoded after receiving. Run the server first, then the client in a second terminal.

TCP exchange: the server waits, the client connects, then they swap bytes



Run the server first: accept() blocks until a client connects.

socket call sequence · server and client

2.2 Client-side scripting

A **client-side script** runs on the user's machine and acts as the client of some server: it opens connections, sends requests and processes replies. It does not wait for requests from others.

```
from urllib.request import urlopen
import json

with urlopen("https://example.com") as page:           # HTTP GET request
    html = page.read().decode("utf-8")
print(html[:80])

# sending an email with the standard library
import smtplib
from email.message import EmailMessage

msg = EmailMessage()
msg["From"] = "me@example.com"
msg["To"] = "friend@example.com"
msg["Subject"] = "Test"
msg.set_content("Hello from Python")

with smtplib.SMTP("smtp.example.com", 587) as smtp:    # placeholder server
    smtp.starttls()
    smtp.login("me@example.com", "app-password")
    smtp.send_message(msg)
```

Typical client-side jobs are downloading files, calling web APIs, checking mailboxes, and GUI programs that talk to a server. The third-party `requests` library is a popular friendlier alternative to `urllib`.

2.3 The PyMailGUI client

PyMailGUI is a full desktop email client written in Python with Tkinter, presented as a case study in *Programming Python*. It shows how the earlier ideas combine in one real program:

- **GUI layer (Tkinter):** windows for the mail list, reading, composing, replying and forwarding, with menus and buttons.
- **Network layer:** `poplib` (or IMAP) downloads messages and `smtplib` sends them.
- **Message layer:** the `email` package parses headers, bodies and attachments and builds new messages.
- **Threads:** slow network operations run in background threads so the window stays responsive, and a queue passes results back to the GUI.
- **Persistence:** local files keep loaded messages and settings.

It is a **client-side** program: all the code runs on the user's computer and the mail server only stores and delivers messages.

2.4 Server-side scripting

A **server-side script** runs on the server in answer to a request from a client, usually a web browser. The server runs the script, which builds a reply (normally HTML or JSON) and sends it back.

	Client-side script	Server-side script
Runs on	The user's computer	The web server
Starts when	The user runs the program	A request arrives
Typical task	Fetch pages, send mail, show a GUI	Build web pages, read a database, enforce rules
Example	PyMailGUI	PyMailCGI

The classic mechanism is **CGI (Common Gateway Interface)**: the web server starts a program for each request, passes the form data in, and returns whatever the program prints. Modern Python web apps use **WSGI** frameworks such as Flask and Django, which keep the application running instead of starting a new process each time.

```
# a tiny server-side program with Flask (pip install flask)
from flask import Flask, request

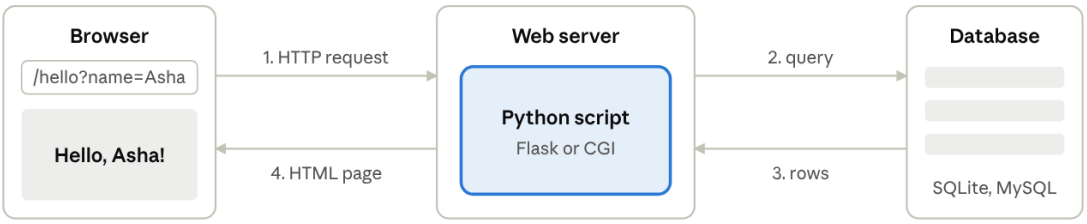
app = Flask(__name__)

@app.route("/hello")                        # URL: /hello?name=Asha
def hello():
    name = request.args.get("name", "guest")
    return f"<h1>Hello, {name}!</h1>"     # HTML sent back to the browser

if __name__ == "__main__":
    app.run(port=8000)
```

The standard `cgi` module that older books use was removed from the standard library in Python 3.13, so use a framework for new work. HTTP is **stateless**: the server forgets each request, so state such as a logged-in user is carried in cookies, hidden form fields or URL parameters.

A server-side script turns each browser request into an HTML reply



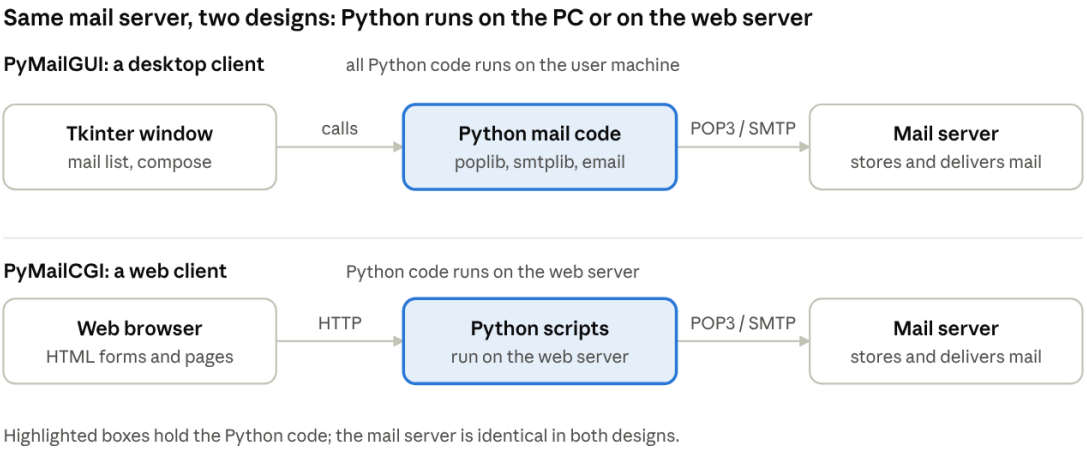
The server forgets each request afterwards (HTTP is stateless), so cookies or hidden fields carry state.

web request cycle · 4 steps

2.5 The PyMailCGI server

PyMailCGI is the web version of the same mail tool, also a case study in *Programming Python*. Instead of a desktop window, the user works in a browser, and Python scripts on the web server do the work.

- The **browser** shows HTML forms and sends each click or submit to the server as an HTTP request.
- The **server-side Python scripts** read the form data, connect to the mail server with `poplib` and `smtplib`, and generate HTML pages as replies.
- Because HTTP is stateless, the scripts must pass the user's session information from page to page (through URLs or hidden form fields) and treat passwords with care.
- Nothing needs installing on the user's computer, which is the main advantage over PyMailGUI; the trade-off is a less responsive interface, because every action is a round trip to the server.



PyMailGUI versus PyMailCGI · 2 designs

3. Tools and Techniques

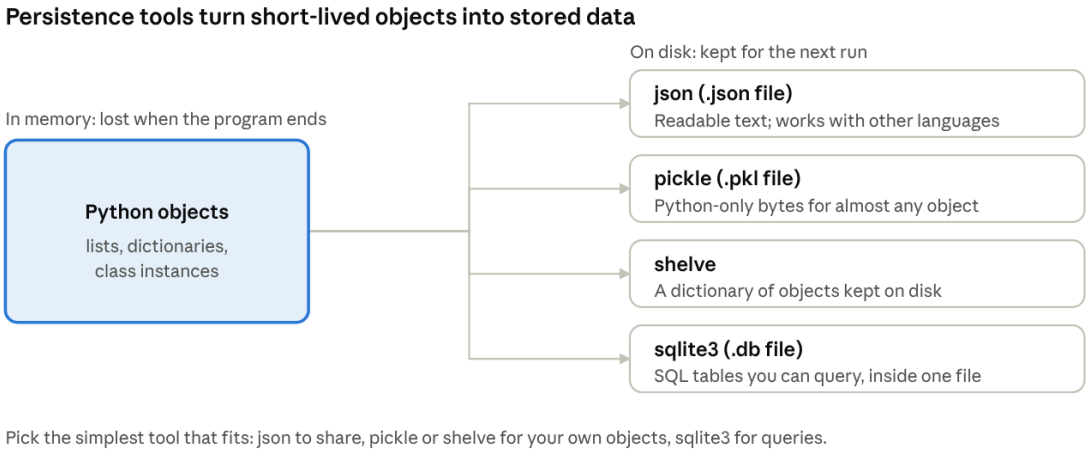
Real programs need to save data between runs, organise it efficiently, process text and sometimes reuse fast C code; this section covers the four tool areas in the syllabus.

3.1 Databases and persistence

Persistence means keeping data after the program ends. Python offers a ladder of options, from saving one object to running a full database.

Tool	What it stores	Best for
`open()` text files	Plain text, CSV	Simple logs and tables
`json`	Lists, dictionaries, numbers, strings as text	Settings and data shared with other languages and web APIs
`pickle`	Almost any Python object as bytes	Saving your own objects quickly (load only trusted files)
`shelve`	A dictionary of pickled objects on disk	A simple key-value store
`dbm`	Byte keys to byte values	Very small key-value files

Tool	What it stores	Best for
`sqlite3`	Tables queried with SQL, inside one file	Structured data and anything needing queries
ORM (SQLAlchemy)	Tables presented as Python classes	Larger applications on MySQL, PostgreSQL and others



persistence tools · 4 options

```
import pickle, shelve, json

student = {"name": "Asha", "marks": [72, 85, 91]}

with open("student.pkl", "wb") as f:
    pickle.dump(student, f)               # save the object
with open("student.pkl", "rb") as f:
    print(pickle.load(f))                 # load it back

with shelve.open("school") as db:          # behaves like a dictionary stored
on disk
    db["asha"] = student
    print(db["asha"]["marks"])

print(json.dumps(student))                 # '{"name": "Asha", "marks": [72,
85, 91]}'
```

Never unpickle data from an untrusted source: loading a crafted pickle can run arbitrary code.

SQLite is a complete SQL database in the standard library; the same SQL works with other database systems through their drivers, which follow the same DB-API pattern of connect, cursor, execute and commit.

```
import sqlite3

conn = sqlite3.connect("school.db")                # creates the file if
missing
```

```
cur = conn.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS students (id INTEGER PRIMARY KEY,
name TEXT, marks INTEGER)")
cur.execute("INSERT INTO students (name, marks) VALUES (?, ?)", ("Asha", 85))
# ? placeholders prevent SQL injection
conn.commit()                                          # make the change
permanent

for row in cur.execute("SELECT name, marks FROM students WHERE marks > ?",
(60,)):
    print(row)                                         # ('Asha', 85)
conn.close()
```

3.2 Data structures

A data structure organises data so that the operations you need are fast. Python's built-in list, tuple, dictionary and set cover most needs; the standard library and your own classes supply the rest.

Structure	Python tool	Operations	Typical use
Stack (last in, first out)	`list` with `append()` and `pop()`	push, pop	Undo, expression evaluation, backtracking
Queue (first in, first out)	`collections.deque`	`append()`, `popleft()`	Task scheduling, breadth-first search
Priority queue	`heapq`	`heappush()`, `heappop()`	Always take the smallest item first
Linked list	Your own `Node` class	insert and delete without shifting	Learning pointers; rarely needed in Python
Tree	Your own `Node` class with children	traverse, search	File systems, decision trees, parsing
Hash table	`dict` and `set`	lookup, insert, delete in near-constant time	Counting, caching, removing duplicates
Graph	A dictionary of lists	traverse, shortest path	Networks and maps

```
from collections import deque
import heapq

stack = []
stack.append(1); stack.append(2)
print(stack.pop())                # 2   (last in, first out)

queue = deque(["a", "b"])
queue.append("c")
print(queue.popleft())            # 'a' (first in, first out)
```

```
heap = []
for n in [5, 1, 3]:
    heapq.heappush(heap, n)
print(heapq.heappop(heap))        # 1   (smallest first)

class Node:                       # building block for linked lists and trees
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

def inorder(node):                # binary tree traversal using recursion
    if node:
        yield from inorder(node.left)
        yield node.value
        yield from inorder(node.right)

root = Node(2); root.left = Node(1); root.right = Node(3)
print(list(inorder(root)))        # [1, 2, 3]
```

↗ stack, queue, linked list and tree · 4 pictures

3.3 Text and language

Text processing is central to data work: cleaning strings, searching with patterns and reading structured formats.

- **String methods:** `split`, `join`, `strip`, `replace`, `find`, `startswith` handle most day-to-day jobs.
- **Regular expressions (`re`):** a compact pattern language for searching and replacing text.
- **Structured text:** `json`, `csv`, `xml.etree.ElementTree` and `html.parser` read common data formats.
- **Unicode:** Python 3 strings hold Unicode text; use `encode("utf-8")` and `decode("utf-8")` at the boundary with files and networks.
- **Command-line languages:** `argparse` reads options such as `--input data.csv`.
- **Parsing a language:** to process your own mini-language you tokenise the text (split it into words and symbols) and then parse the tokens by rules. Python can also run text as code with `eval()` and `exec()`, but never do that with untrusted input.

```
import re, json

text = "Contact: asha@example.com, ravi@test.org on 2026-10-09"
print(re.findall(r"[\w.]+@[\w.]+\.\w+", text))      # ['asha@example.com',
'ravi@test.org']
print(re.search(r"(\d{4})-(\d{2})-(\d{2})", text).groups())   # ('2026',
'10', '09')
print(re.sub(r"\d", "#", "Room 204"))               # 'Room ###'

raw = '{"city": "Kochi", "temp": 31}'
data = json.loads(raw)                               # text -> dictionary
print(data["city"])

def tokenize(expr):                                  # a tiny tokenizer
    return re.findall(r"\d+|[+\-*/()]", expr)
```

```
print(tokenize("12 + 3*(4 - 1)"))                    # ['12', '+', '3', '*',
'(', '4', '-', '1', ')']
```

3.4 Python/C integration

Python is easy to write but slower than C for heavy number crunching, and a huge amount of useful software already exists as C libraries. Integration lets the two work together in one of two directions.

- **Extending:** call C code *from* Python. You write a C extension module, or use a bridge, and then simply `import` it. NumPy, pandas and many other libraries work this way.
- **Embedding:** call Python *from* C. A C program starts the Python interpreter inside itself so users can script the application.

Approach	How it works	Effort
`ctypes`	Load a compiled shared library and call its functions from pure Python	Low
`cffi`	Describe the C functions in C-like declarations and call them	Low to medium
Cython	Write Python-like code with C types; it compiles to a C extension	Medium
C extension module (`Python.h`)	Write C functions that follow the Python C API	High, but fastest and most flexible
`subprocess`	Run a separate C program and read its output	Low, but slower

↗ extending and embedding · 2 lanes

```
# ctypes: call the C maths library from Python (Linux or macOS)
import ctypes
from ctypes.util import find_library

libm = ctypes.CDLL(find_library("m"))
libm.cos.argtypes = [ctypes.c_double]     # declare the C types
libm.cos.restype = ctypes.c_double
print(libm.cos(0.0))                      # 1.0
```

```
/* fastmath.c : a minimal C extension module */
#include <Python.h>

static PyObject* add(PyObject* self, PyObject* args) {
    int a, b;
    if (!PyArg_ParseTuple(args, "ii", &a, &b)) return NULL;   /* read 2 ints
*/
    return PyLong_FromLong(a + b);                            /* return a
Python int */
```

```
}

static PyMethodDef Methods[] = {
    {"add", add, METH_VARARGS, "Add two integers"},
    {NULL, NULL, 0, NULL}
};

static struct PyModuleDef mod = {PyModuleDef_HEAD_INIT, "fastmath", NULL, -1,
Methods};

PyMODINIT_FUNC PyInit_fastmath(void) { return PyModule_Create(&mod); }
```

After building the module (for example with `setuptools`), Python code simply runs `import fastmath` and then `fastmath.add(2, 3)`, which returns `5`.

Quick revision

Remember one core idea for each topic.

- **GUI options:** Tkinter ships with Python; PyQt, wxPython and Kivy are installed separately.
- **Widgets and layout:** create widgets inside a parent, place them with `pack`, `grid` or `place` (one manager per container).
- **Event loop:** `mainloop()` waits for events and runs your callbacks; keep callbacks short.
- **Coding techniques:** classes, `bind()`, control variables, `after()` timers and worker threads with a queue.
- **Customizing:** options such as `font`, `bg` and `relief`, `ttk` themed widgets, images and subclassed widgets.
- **Network scripting:** sockets carry bytes; a server does bind, listen, accept; a client does connect, send, receive.
- **Client-side versus server-side:** client-side runs on the user's machine and requests services; server-side runs on the server in answer to requests.
- **PyMailGUI:** a Tkinter mail client that combines GUI, `poplib`, `smtplib`, `email` and threads.
- **PyMailCGI:** a browser-based mail client where server-side scripts do the work and state is passed between pages because HTTP is stateless.
- **Persistence:** `json` for portable data, `pickle` and `shelve` for Python objects, `sqlite3` for SQL.
- **Data structures:** list as stack, `deque` as queue, `heapq` as priority queue, `dict` and `set` as hash tables, classes for trees and linked lists.
- **Text and language:** string methods, `re` patterns, `json` and `csv` parsing, tokenise then parse.
- **Python/C integration:** extending calls C from Python (`ctypes`, Cython, C extension modules); embedding runs Python inside a C program.

Likely exam questions

Short answer (2 to 3 marks)

1. Name any four Tkinter widgets and state the use of each.
2. Differentiate between `pack`, `grid` and `place`.
3. What is a callback function in a GUI program?
4. What is a socket? Which module provides it in Python?
5. Differentiate between client-side and server-side scripting.

6. What is the difference between `pickle` and `json`?
7. What is the difference between extending and embedding Python?

Long answer (5 to 10 marks)

1. Explain the structure of a Tkinter program with a suitable example, covering widgets, layout and the event loop.
2. Explain how GUI widgets can be customised, with examples.
3. Describe how a TCP client and server communicate using sockets, with a program.
4. Explain the working of PyMailGUI and PyMailCGI and compare the two designs.
5. Explain the options for persistence of data in Python, with examples of `pickle`, `shelve` and `sqlite3`.
6. Describe stack, queue and tree implementations in Python.
7. Explain the ways of integrating Python with C code.

References

Text books

1. Mark Lutz, *Learning Python*, O'Reilly.
2. Mark Lutz, *Programming Python*, O'Reilly (GUI programming, Internet programming, PyMailGUI, PyMailCGI, databases and persistence, data structures, text and language, and Python/C integration).
3. Wes McKinney, *Python for Data Analysis*, O'Reilly.

Reference books

1. Martin C. Brown, *Python: The Complete Reference*, McGraw Hill Education.
2. Paul Barry, *Head First Python*, O'Reilly.